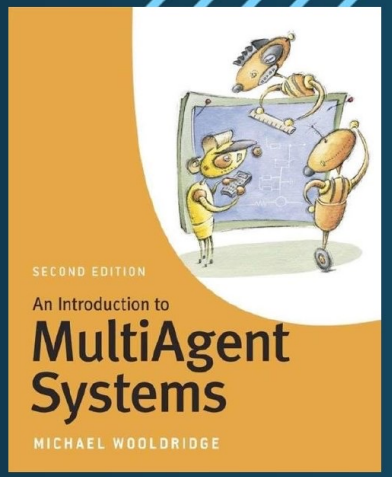
Princess Nourah Bint Abdulrahman University

GUEST LECTURE | HIGH-IMPACT ACADEMIC PRACTICES | PNU × UTM

Performance Evaluation of Agent-Based Systems



CAI 461 · Topic 8 — from rational agents to LLM multi-agent systems

Prof. Dr. Mohd Shahizan Othman
Universiti Teknologi Malaysia (UTM)

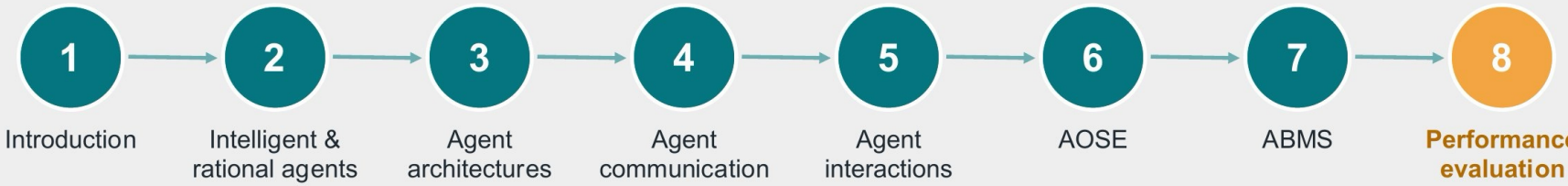
Host: Dr. Eman Aljabarti — Course Coordinator, CAI 461, PNU

جامعة الأميرة نورة بنت عبدالرحمن
كلية علوم الحاسب والمعلومات

Session Plan

0	Opening & where we are	6 min
1	What does “performing well” mean?	15 min
2	Evaluating a society of agents	18 min
3	Measuring the system: metrics, tracing, experiments	15 min
4	Evaluating agent-based models (ABMS)	12 min
5	New models for evaluating LLM-based agents	22 min
6	Case activity: design an evaluation plan	12 min
7	Lab preview, takeaways, Q&A	10 min

Where We Are in CAI 461



Today's question re-uses every earlier topic:

Topic 2	performance measure, rationality, utility
Topics 3–4	what to log inside an agent and in its messages
Topic 5	social welfare, Pareto efficiency, Nash stability
Topic 7	verification, validation, sensitivity analysis

Four-layer Evaluation Framework

From individual agent performance to trustworthy deployment

1 Individual agent

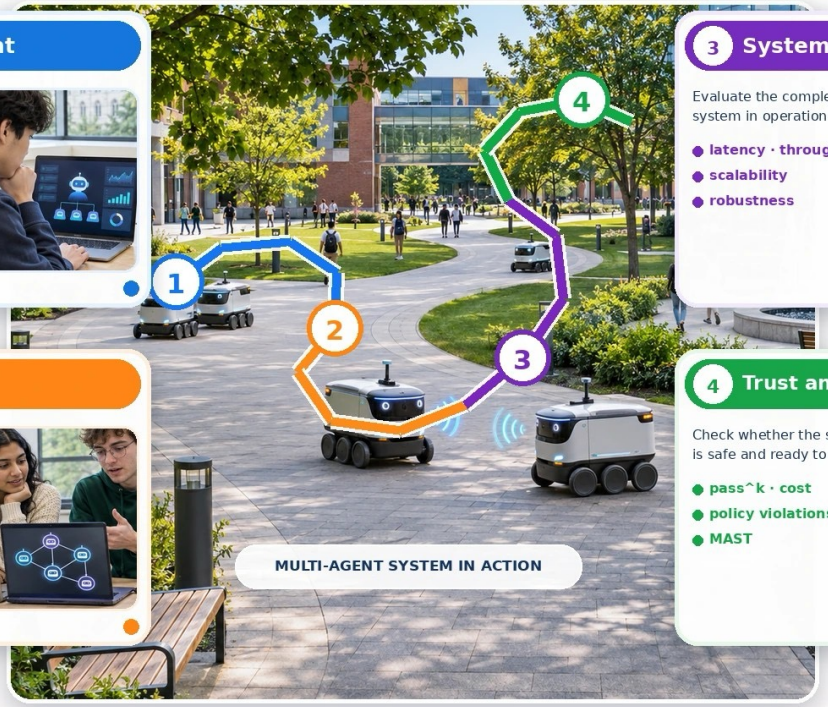
Measure how one agent performs toward its objective.

- performance measure
- EU
- P(Ψ)

2 Interaction

Assess group outcomes and coordination among agents.

- welfare · Pareto
- stability · PoA
- messages



3 System

Evaluate the complete system in operation.

- latency · throughput
- scalability
- robustness

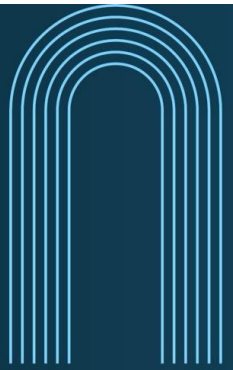
4 Trust and deployment

Check whether the system is safe and ready to use.

- pass^k · cost
- policy violations
- MAST

Evaluate each layer with metrics that match its purpose.

Princess Nourah Bint
Abdulrahman University



Part 1

What Does “Performing Well” Mean?

Performance measures, utility and task specifications

What Does 'Good Performance' Mean?

Define success before judging an agent

1. Choose the Right Measure

Measure the outcomes you want in the environment—not the agent's activity.

Example:
Vacuum: dirt collected is a weak measure. Better: clean squares over time, minus energy and noise.

2. Rational Is Not Perfect

- **Omniscient:** knows actual outcomes—impossible.
- **Rational:** maximizes expected performance with available information.
- **Bounded-optimal:** the best program that can run on the machine.

Rationality depends on the performance measure, prior knowledge, available actions, and percepts.

3. Compare Expected Utility

$$EU = \sum u(r) \times P(r)$$

Example:
Agent A: 0.7 × 10 = 7
Agent B: 0.9 × 7 = 6.3

By this utility, Agent A is better.

4. Specify the Task

- **Achievement:** reach goal G at least once.
- **Maintenance:** always avoid bad state B.
- **Success probability:** $P(\Psi \mid \text{Agent, Environment})$

The Performance Measure

- A performance measure evaluates the sequence of environment states that the agent's actions produce.
- It is chosen by the designer, not by the agent.
- **Design rule: measure what you actually want in the environment, not how you think the agent should behave.**

Example: the vacuum agent

Bad measure: "amount of dirt picked up".
A rational agent can clean, dump the dirt back, and clean again — scoring very high.

Better measure: "number of clean squares at each time step", minus a penalty for energy and noise.

Source: Russell & Norvig (2020), Artificial Intelligence: A Modern Approach, 4th ed., Ch. 2, §2.2 "Good Behavior: The Concept of Rationality".

↩ Link back: Lecture 2 — PEAS: the "P" is the performance measure

Rational Is Not the Same as Perfect

A rational agent selects the action that is expected to maximize its performance measure, given:

Performance measure	Prior knowledge	Available actions	Percept sequence
defines success	of the environment	what it can do	what it has seen so far

Omniscient	knows the real outcome of every action	**Impossible to build**
Rational	maximizes expected performance with the information it has	**Our design target**
Bounded-optimal	the best program that can actually run on the given machine	**Realistic target**

Source: Russell & Norvig (2020), Ch. 2 §2.2; Wooldridge (2009), An Introduction to MultiAgent Systems, 2nd ed., Ch. 2 (bounded optimal agents, after Russell & Subramanian, 1995).

From Measure to Number: Utility Functions

- A run r is a sequence of environment states and agent actions.
- A utility function u assigns a real number to every run: $u : R \rightarrow \mathbb{R}$.
- Because environments are uncertain, we evaluate an agent by its expected utility over all runs.
- The optimal agent maximizes expected utility.

Expected utility of agent Ag in environment Env

$$EU(Ag, Env) = \sum_{r \in R(Ag, Env)} u(r) \cdot P(r \mid Ag, Env)$$

$$Ag_opt = \arg\max_{Ag \in AG} EU(Ag, Env)$$

Worked example — Agent A reaches the goal (u = 10) with P = 0.7, else fails (u = 0): EU = 7. Agent B succeeds with P = 0.9 but takes a longer route (u = 7): EU = 6.3. By this utility, A is better — change u, and the answer can change.

Source: Wooldridge (2009), Ch. 2 "Intelligent Agents" — utility functions over runs and optimal agents.

Achievement and Maintenance Tasks

Often we only care whether a run succeeds or fails. A predicate task specification $\Psi(r)$ returns true or false for each run.

Achievement task — "reach G"

- Success if some state in the set G is reached.
- Vacuum: "all squares clean".
- Delivery robot: "parcel delivered to Room 2B".

Maintenance task — "always avoid B"

- Success if no state in the set B ever occurs.
- Vacuum: "never collide with a person".
- Data-centre agent: "temperature never above 30 °C".

Metric that follows directly: $P(\Psi \mid Ag, Env)$ — the probability that the agent succeeds. In practice we estimate it as the success rate over many runs.

Source: Wooldridge (2009), Ch. 2 — predicate task specifications; achievement and maintenance tasks.

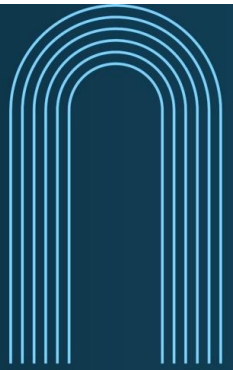
Quick Check (2 minutes)

Classify each requirement: achievement (A) or maintenance (M)?

1	A drone agent must photograph all 12 solar panels on the roof.	A / M
2	A trading agent must never hold more than 20% of the budget in one stock.	A / M
3	A tutoring agent must keep each student's answers private at all times.	A / M
4	A meeting-scheduler agent must find a slot that all five members accept.	A / M

Answer in the chat or the poll. Answers are in the speaker notes.

Princess Nourah Bint
Abdulrahman University



Part 2

Evaluating a Society of Agents

Social welfare, Pareto efficiency, stability, price of anarchy, protocol criteria

Why Is Multi-Agent Evaluation Difficult?

Criteria for Judging Group Outcomes

Criterion	Question it answers	How to compute
Utilitarian social welfare	Is the total benefit as large as possible?	$sw(\omega) = \Sigma_i u_i(\omega)$
Egalitarian social welfare	How well off is the worst-off agent? (fairness)	min over i of $u_i(\omega)$
Pareto efficiency	Can anyone be made better off without making someone worse off?	ω is efficient if no ω' is at least as good for all and better for one
Stability (Nash equilibrium)	Does any agent want to deviate on its own?	No agent gains by changing only its own strategy
Individual rationality	Is every agent better off participating than staying out?	$u_i(\omega) \geq$ utility of not participating

Source: Wooldridge (2009), Ch. 11 "Multiagent Interactions"; Russell & Norvig (2020), Ch. 18 "Multiagent Decision Making"; Shoham & Leyton-Brown (2008), Ch. 3.

↩ Link back: Agent Interactions lecture — competition & game theory

Worked Example: When Stable Is Not Efficient

Prisoner's Dilemma (payoffs: row agent, column agent)

	j cooperates	j defects
i cooperates	**3 , 3** sw = 6 ✓ Pareto	**0 , 5** sw = 5
i defects	**5 , 0** sw = 5	**2 , 2** sw = 4 ✓ Nash

- (Defect, Defect) is the only Nash equilibrium — it is stable.
- But it is not Pareto efficient: (Cooperate, Cooperate) makes both agents better off.
- Social welfare: 4 at the equilibrium versus 6 at the best outcome.
- **Lesson for evaluation: a protocol can be stable and still waste value. Report both stability and welfare.**

Source: Wooldridge (2009), Ch. 11 — the Prisoner's Dilemma (Wooldridge's payoff values).

↩ Link back: Agent Interactions lecture — dominant strategies & Nash equilibrium

Price of Anarchy: the Cost of Selfishness

$$\text{PoA} = \frac{\text{cost of worst equilibrium}}{\text{cost of optimal outcome}}$$

- PoA = 1: selfish agents reach the best outcome.
- PoA > 1: decentralization has a price.
- Use it to decide whether a central coordinator is worth its overhead.

Example: Pigou's routing network (one unit of traffic)

Selfish (Nash): everyone takes Road 2 → average cost 1
Optimal: half on each road → average cost ¾
PoA = 1 ÷ ¾ = 4/3 ≈ 1.33

Source: Shoham & Leyton-Brown (2008), Ch. 6, §6.4 "Selfish routing and the price of anarchy"; Koutsoupias & Papadimitriou (1999), STACS.

When Is a Society of Agents Performing Well?

1 → 2 → 3

1. Judge the Group Outcome

- **Utilitarian welfare** — maximise total benefit
- **Egalitarian welfare** — protect the worst-off agent
- **Pareto efficiency** — no one can improve without someone else losing
- **Nash stability** — no agent gains by changing strategy alone
- **Individual rationality** — participation beats opting out

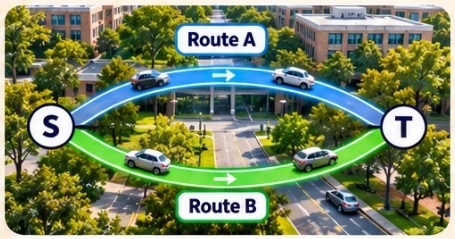
2. Stable Is Not Always Efficient

Prisoner's Dilemma (payoffs: you, other)

	Cooperate	Defect
Cooperate	3,3 · Welfare 6 · Pareto efficient	0,5
Defect	5,0	2,2 · Welfare 4 · Nash equilibrium

Takeaway: *The stable outcome gives lower group welfare.*

3. Measure the Cost of Selfishness



$$\text{Price of Anarchy} = \frac{\text{worst equilibrium cost}}{\text{optimal cost}}$$

- **Selfish equilibrium:** average cost 1
- **Optimal routing:** average cost 0.75

$$\text{Price of Anarchy} = \frac{1}{0.75} = \frac{4}{3} \approx 1.33$$

Report welfare and stability together.

Evaluating a Protocol or Mechanism

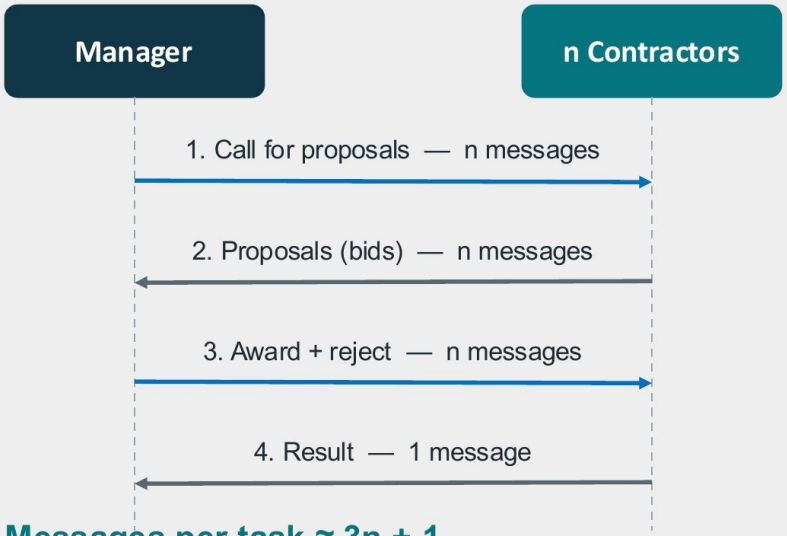
1. **Guaranteed success**
   Does the protocol always reach an agreement?
2. **Maximizes social welfare**
   Is the agreed outcome the best for the group?
3. **Pareto efficiency**
   Is no value left on the table?
4. **Individual rationality**
   Do agents gain by following the protocol?
5. **Stability**
   Do agents have an incentive to follow it honestly?
6. **Simplicity**
   Is computation and communication low for each agent?
7. **Distribution**
   Can it run without a single point of failure?

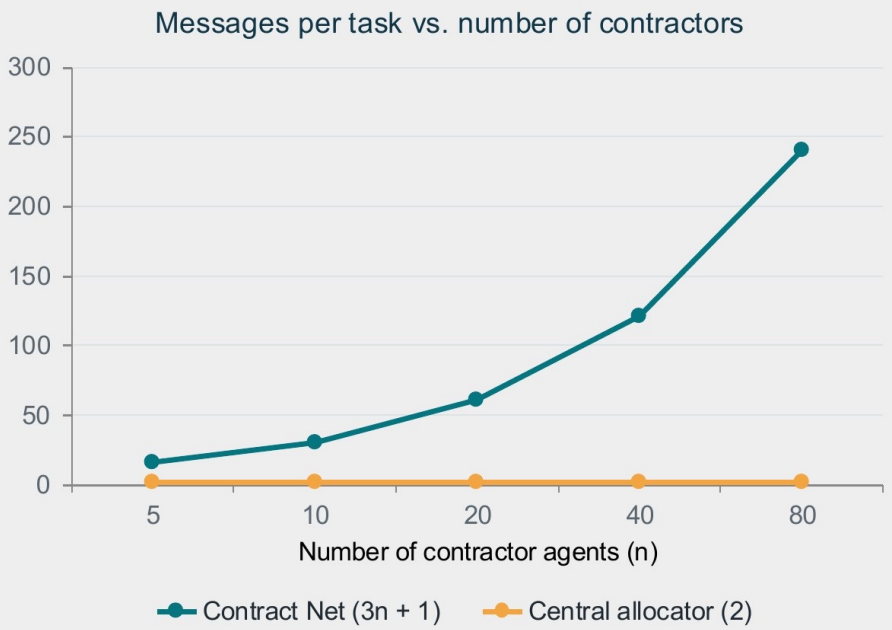
Use this list as a checklist when you compare auctions, voting or negotiation protocols in a report.

Source: Wooldridge (2009), Part III "Multiagent Decision Making" (protocol properties, after Rosenschein & Zlotkin, 1994 and Sandholm, 1999).

↩ Link back: Agent Interactions lecture — auctions & negotiation

Worked Example: Measuring Contract Net





Messages per task ≈ 3n + 1

Also measure: allocation cost vs. optimal, time to award, and success when a contractor fails.

Source: Smith (1980), IEEE Trans. Computers — the Contract Net Protocol; Wooldridge (2009), Ch. 8 "Working Together". Message counts derived from the protocol steps.

↩ Link back: Agent Interactions lecture — task sharing with Contract Net

Is a Multi-Agent Protocol Worth Using?

Weigh outcome quality against coordination cost

PROTOCOL QUALITY CHECK

- Guaranteed success
- Maximises social welfare
- Pareto efficiency
- Individual rationality
- Stability
- Simplicity: low computation and communication
- Distribution: no single point of failure

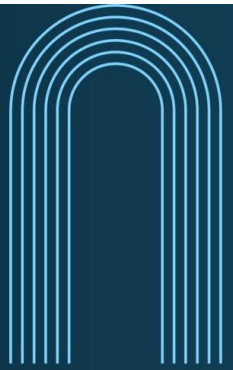
CONTRACT NET: MESSAGES PER TASK

Contract Net ≈ 3n + 1 messages

Central allocator ≈ 2 messages

ALSO MEASURE | Allocation cost vs. optimum • Time to award • Success when a contractor fails

Princess Nourah Bint
Abdulrahman University



Part 3

Measuring the System

Metric families, instrumentation & tracing, sound experiments

How Do We Measure a Multi-Agent System?

Six families of performance metrics

Effectiveness

Task success · Solution quality · Social welfare

Efficiency

Latency · Throughput · CPU, memory, tokens, and cost

Fairness & Trust

Worst-off agent · Run-to-run consistency · Safety violations

Coordination

Messages per task · Conflicts · Duplicated work · Waiting time

Robustness

Test agent failures and message loss · Measure recovery time

Scalability

Measure as agents or tasks grow · Find the breaking point

Six Families of MAS Performance Metrics

Effectiveness

- Task success rate
- Solution quality vs. optimal
- Social welfare

Efficiency

- Latency / time to complete
- Throughput (tasks per minute)
- CPU, memory, tokens, cost

Coordination

- Messages per task
- Conflicts and duplicated work
- Idle or waiting time

Scalability

- How each metric grows with the number of agents or tasks
- Breaking point

Robustness

- Success when agents fail or messages are lost
- Recovery time

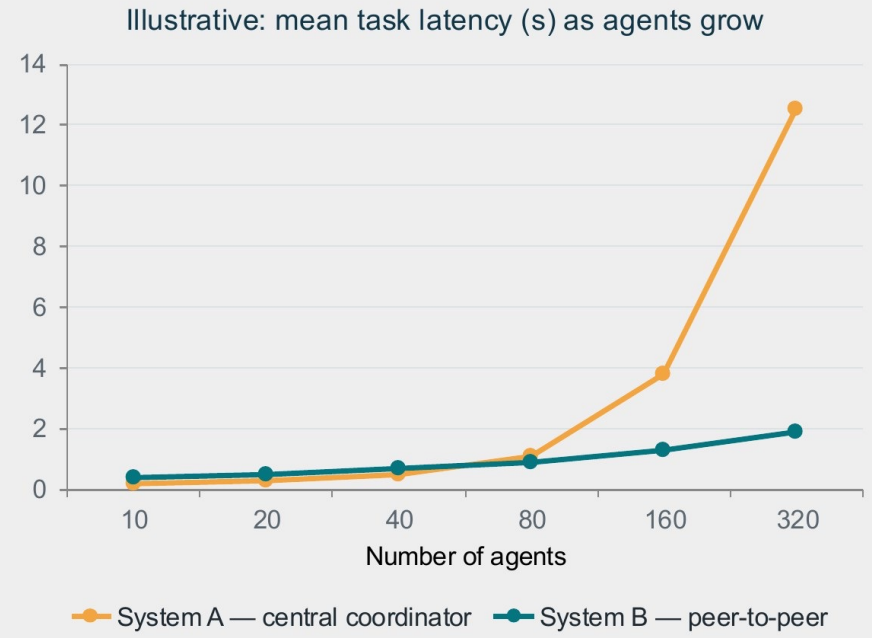
Fairness & trust

- Egalitarian welfare
- Consistency across repeated runs
- Policy or safety violations

Source: Synthesized from Wooldridge (2009) Ch. 8 & 11, Bellifemine, Caire & Greenwood (2007) Ch. on JADE tools, and Kapoor et al. (2025), TMLR.

Scalability & Robustness: Curves, Not Points

- Run the same system while increasing one factor: number of agents, tasks or message delay.
- Plot each metric against that factor; look for the knee where performance breaks.
- Robustness test: remove k agents (or drop p% of messages) during a run and record success rate and recovery time.
- **Report the curve and the breaking point, not just one good run.**



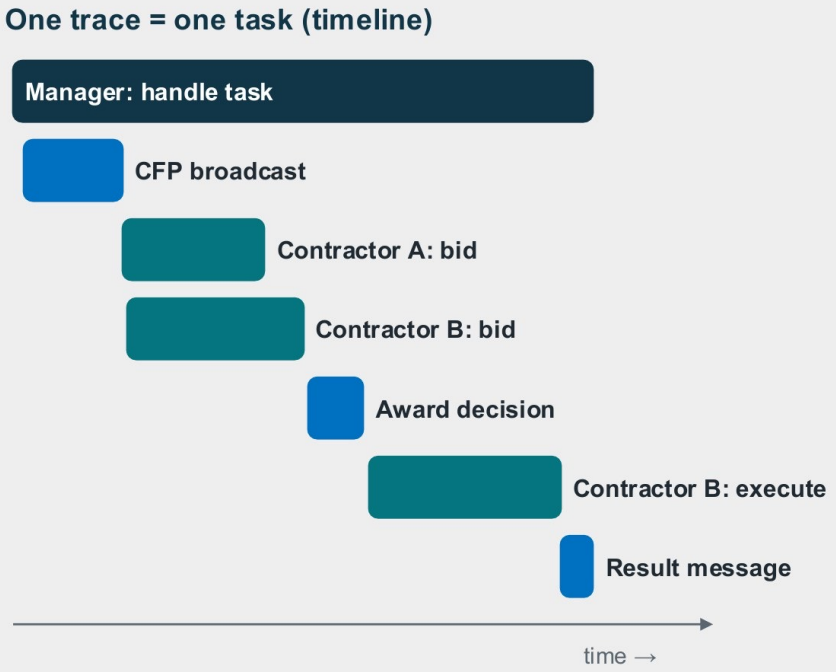
Values are illustrative, to show how to read a scalability curve.

↩ Link back: Agent Interactions lecture — centralized vs. distributed coordination

Instrumentation and Tracing

- Instrumentation: add code that records events — perceptions, decisions, actions, messages, errors — with timestamps.
- Trace: the full story of one task across all agents; each step is a span with a start time, end time and parent.
- Correlation IDs link the steps of one conversation across agents.

FIPA-ACL already gives you correlation fields:
`conversation-id` · `reply-with` · `in-reply-to` · `protocol`



Source: FIPA (2002), ACL Message Structure Specification SC00061G; Bellifemine, Caire & Greenwood (2007), Developing Multi-Agent Systems with JADE.

↩ Link back: Agent Communication lecture — ACL message parameters

Monitoring Agents on the JADE Platform

Sniffer Agent

Draws the messages exchanged between selected agents as a sequence diagram — ideal for checking protocols and counting messages.

Introspector Agent

Shows the internal state of one agent: its behaviours and its incoming and outgoing message queues.

Remote Monitoring Agent (RMA)

The platform console: containers, agents, life-cycle (start, suspend, kill) — useful for robustness tests.

Log Manager Agent

Changes logging levels at run time, so you can collect detailed logs without restarting.

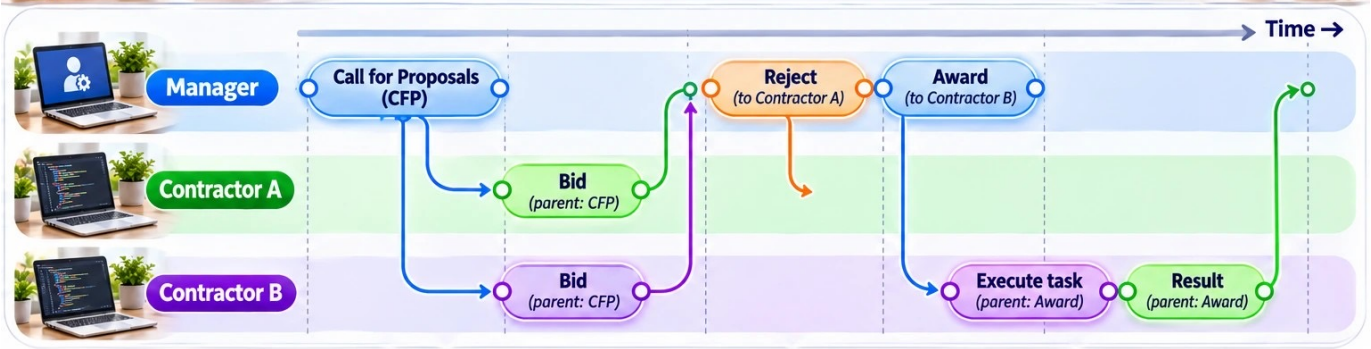
Source: Bellifemine, Caire & Greenwood (2007), Developing Multi-Agent Systems with JADE, Wiley — chapter on JADE tools.

↩ Link back: Communication lab — messaging on a standards-based platform

From Messages to Traces: Observing a Multi-Agent System

Follow one task across agents, tools, and time

Instrumentation records perceptions, decisions, actions, messages, and errors with timestamps.



Trace explanation

- One trace = the full story of one task across all agents.
- Each step is a span with a start time, end time, and parent.

FIPA-ACL correlation fields: conversation-id · reply-with · in-reply-to · protocol

JADE OBSERVABILITY TOOLS

Sniffer
message sequence; inspect protocols and count messages.

Introspector
one agent's behaviours and message queues.

RMA
platform containers, agents, and lifecycle.

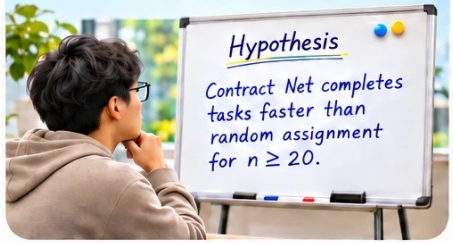
Log Manager
change logging levels at run time.

How to Design a Sound Evaluation Experiment

A practical six-step checklist for multi-agent systems

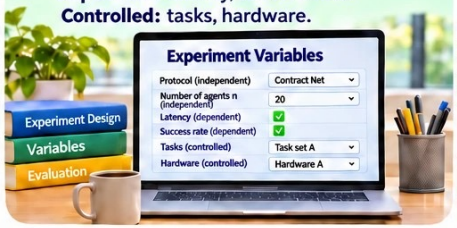
1 State a hypothesis

Make a specific, testable prediction.



2 Define variables

Independent: protocol, number of agents (*n*).
Dependent: latency, success rate.
Controlled: tasks, hardware.

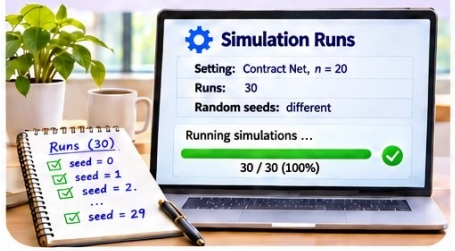


3 Choose a baseline

Compare with random assignment, a greedy method, or a single agent.

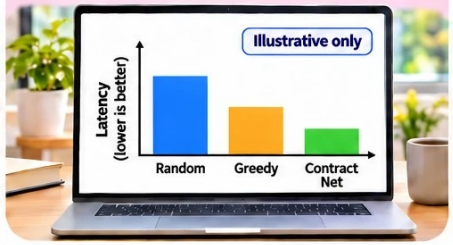
4 Repeat the runs

Run each setting 30 times with different random seeds.



5 Report statistics

Give mean ± standard deviation or a 95% confidence interval; test for differences.



6 Enable reproduction

Publish the code, parameters, seeds, and model versions.

EXAMPLE HYPOTHESIS | Contract Net completes tasks faster than random assignment for $n \geq 20$.

Princess Nourah Bint
Abdulrahman University

Part 4

Evaluating Agent-Based Models

Verification, validation, replication, sensitivity analysis, ODD

~12 minutes

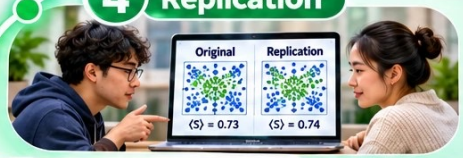
How Do We Test an Agent-Based Model?

Build confidence through checks, experiments, and transparent reporting

5 Report with ODD

- **Overview:** purpose, entities, process.
- **Design concepts:** emergence, adaptation, objectives, interaction, stochasticity.
- **Details:** initialization, inputs, submodels.

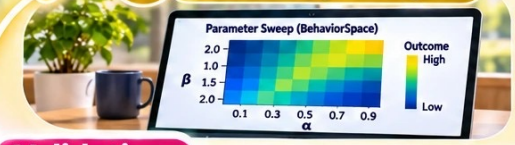
4 Replication



Can someone else get the same results?

Independently re-implement the model and compare outputs statistically.

3 Sensitivity Analysis



Vary parameters; repeat stochastic runs.

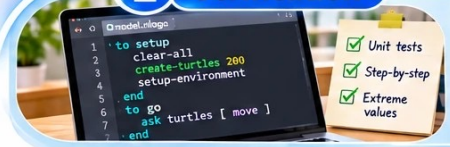
Find which assumptions strongly change the results. NetLogo BehaviorSpace can automate sweeps.

2 Validation

Did we build the right model?

Check realistic individual behaviour, system-level patterns, and compare with real data.

1 Verification



Did we build the model right?

Check code against the conceptual model. Use unit tests, step-by-step runs, and extreme values.

Verification, Validation and Replication

Verification

Did we build the model right?

- Does the code match the conceptual model?
- Unit tests, step-by-step runs, extreme parameter values.

Validation

Did we build the right model?

- Micro-face: do individual agents behave realistically?
- Macro-face: do overall patterns look like reality?
- Empirical: compare outputs with real data.

Replication

Can someone else get the same results?

- Re-implement the model independently and compare outputs statistically.
- The strongest test of both V and V.

Source: Wilensky & Rand (2015), Ch. 7 "Verification, Validation, and Replication"; Wilensky & Rand (2007), JASSS 10(4):2.

↩ Link back: ABMS lecture & Project 1 (NetLogo)

Sensitivity Analysis and Parameter Sweeps

- Change one parameter at a time across a range; keep others fixed (one-factor-at-a-time).
- Run each setting many times; agent-based models are stochastic.
- Look for parameters where small changes cause large output changes — these need the most careful justification.
- In NetLogo, BehaviorSpace automates sweeps and repetitions and exports results to CSV.

Example experiment design — NetLogo Wolf Sheep Predation

Wolf gain from food	Runs	Measured output
10	30	% of runs where wolves survive 500 ticks
20	30	Mean and SD of sheep population
30	30	Period of population cycles
40	30	Ticks until one species goes extinct

4 settings × 30 runs = 120 simulations in one BehaviorSpace experiment.

Source: Wilensky & Rand (2015), Ch. 6 "Analyzing Agent-Based Models" (BehaviorSpace, sensitivity analysis); Wilensky (1997), NetLogo Wolf Sheep Predation model.

↩ Link back: ABMS lab — constructing, simulating and analyzing models

Reporting Models: the ODD Protocol

O

Overview

- Purpose and patterns
- Entities, state variables, scales
- Process overview and scheduling

D

Design concepts

- Emergence, adaptation, objectives
- Sensing, interaction, stochasticity
- Observation: what is measured

D

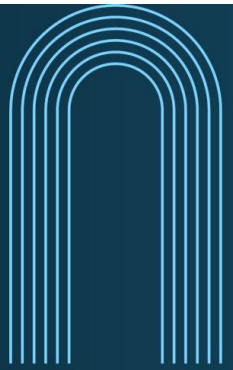
Details

- Initialization
- Input data
- Submodels

ODD is the most widely used standard for describing agent-based models; it makes replication and peer evaluation possible.

Source: Grimm et al. (2006), Ecological Modelling 198:115–126; Grimm et al. (2020), JASSS 23(2):7 "The ODD protocol … second update".

Princess Nourah Bint
Abdulrahman University



Part 5

New Models for Evaluating LLM-Based Agents

Benchmarks, reliability, cost, failure taxonomies, LLM-as-a-judge, observability

From 1980 to Today

A timeline of agent protocols, models and evaluation

1 Foundations

1980 | Contract Net Protocol
Smith publishes task sharing by announce, bid and award.

Used today: Messages per task and allocation quality. Module 2

1995 | Bounded optimality
Russell and Subramanian define the best agent that can actually run on a given machine.

Used today: The realistic target for a rational agent. Module 1

1999 | Price of anarchy
Koutsoupias and Papadimitriou measure how much selfish behaviour costs a system.

Used today: Deciding whether a coordinator is worth it. Module 2

2 Agent communication

2002 | FIPA ACL message structure
The standard adds conversation-id, reply-with and in-reply-to to every agent message.

Used today: Correlation IDs in today's traces. Module 3

2006 | ODD protocol
Grimm and colleagues standardise how agent-based models are described. Updated in 2020.

Used today: Replicable simulation reports. Module 4

3 Evaluation foundations

2009 | Utility over runs
Wooldridge's second edition formalises expected utility and task predicates for agents.

Used today: Every success rate on every benchmark. Module 1

4 LLM evaluation and benchmarks

2023 | LLM-as-a-judge
Zheng and colleagues show strong models agree with human preferences over 80% of the time, and document their biases.

Used today: Scoring open-ended outputs at scale. Module 5

2024 | Agent benchmarks
AgentBench, GAIA and SWE-bench appear at ICLR. "AI Agents That Matter" argues for reporting cost.

Used today: Standard tests and cost-aware comparison. Module 5

5 Reliability and scale

2025 | Reliability and failure
τ-bench introduces pass^k, MultiAgentBench compares topologies, and MAST names 14 failure modes.

Used today: Layer 4: trust and deployment. Module 5

2026 | Holistic leaderboards
HAL runs many benchmarks with one harness and reads agent logs at scale.

Used today: Third-party, cost-aware evaluation. Module 5

Why LLM Agents Need New Methods

Classic agent	LLM-based agent
Actions from a fixed, known set	Free-text reasoning plus open tool calls
Mostly deterministic given a seed	Stochastic: the same input can give different runs
Success checked by a simple test	Output is often text — correctness is hard to check
Cost ≈ CPU time	Cost = tokens and money for every step
Messages in a formal ACL	Messages in natural language — easy to misunderstand

The goal is still Wooldridge's: estimate expected utility and success probability — but with new instruments.

↩ Link back: Agent Architectures lab — LLM agents with tools

The Benchmark Landscape for LLM Agents

Benchmark	What it tests	How success is measured	Venue
AgentBench	LLM-as-agent in 8 environments: OS, database, web, games…	Task-specific success scores	ICLR 2024
GAIA	466 real-world assistant questions needing tools and browsing	Exact final answer; humans ≈ 92% vs. GPT-4 with plugins ≈ 15%	ICLR 2024
SWE-bench	2,294 real GitHub issues from 12 Python repositories	Generated patch must pass the project's unit tests	ICLR 2024
τ-bench	Agent + simulated user + tools + policy (retail, airline)	Final database state matches the goal; pass^k reliability	ICLR 2025
MultiAgentBench	Collaboration and competition among LLM agents	Task score + milestone KPIs + coordination score	ACL 2025

Source: Liu et al. (2024); Mialon et al. (2024); Jimenez et al. (2024); Yao et al. (2025); Zhu et al. (2025) — full references at the end.

Reliability: pass@k vs. pass^k

pass@k — at least one success

Probability that at least one of k attempts succeeds. Good for "try many, keep the best" (e.g., code generation).

pass^k — all k succeed

Probability that all k independent trials of the same task succeed. Measures consistency — what a real customer experiences.

Reported finding: a GPT-4o function-calling agent reached about 61% pass^1 in τ-retail, but pass^8 fell below 25%.

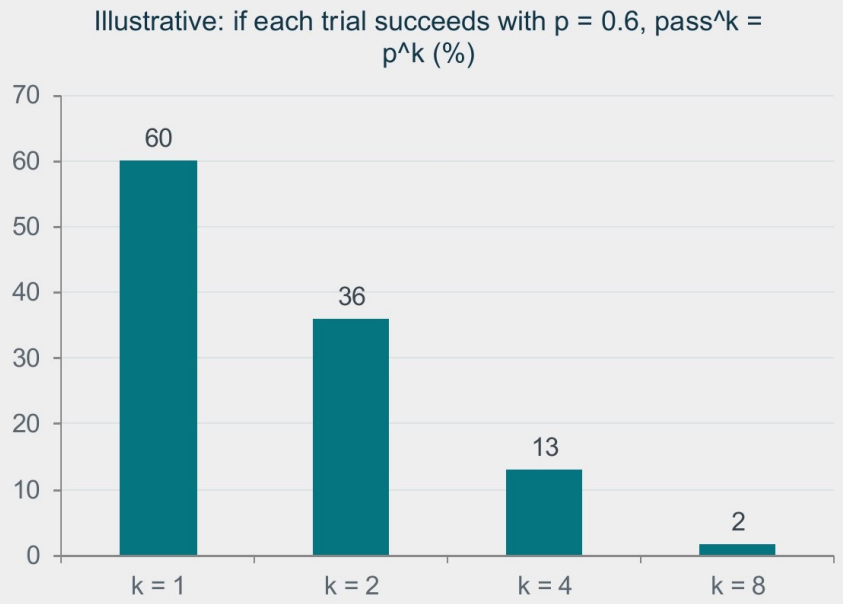


Illustration assumes independent trials; real agents are often worse because failures repeat on the same hard tasks.

Source: Yao, Shinn, Razavi & Narasimhan (2025), "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", ICLR 2025.

↩ Link back: Part 1 — success probability P(Ψ | Ag, Env)

Accuracy Is Not Enough: Report Cost Too

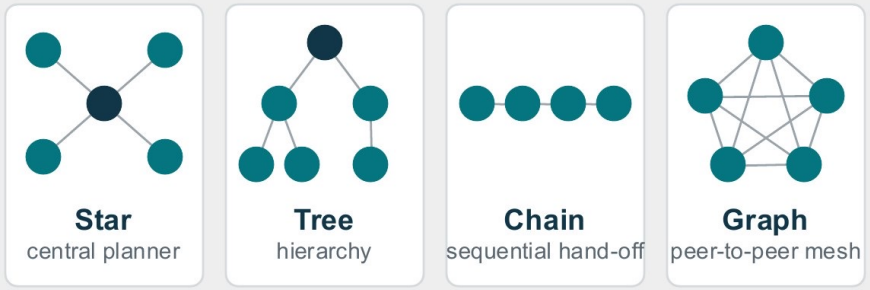
- Many agent papers report only accuracy; a complex agent may cost 10–100× more for a small gain.
- Kapoor et al. recommend plotting accuracy against cost and keeping only agents on the Pareto frontier.
- Simple baselines (retry, majority vote) often match complex agent architectures at lower cost.
- Hold-out test sets prevent agents that “overfit” the benchmark.

Points at the upper-left are best. Agents below and to the right of others are dominated.

Source: Kapoor, Stroebl, Siegel, Nadgir & Narayanan (2025), “AI Agents That Matter”, Transactions on Machine Learning Research (TMLR).

↩ Link back: Part 2 — Pareto efficiency, now applied to accuracy and cost

MultiAgentBench: Evaluating LLM Teams



Coordination topologies compared

Task Score	quality of the final output
Milestone KPI	which sub-goals were reached, and each agent's contribution
Coordination Score	communication + planning quality, rated 1–5

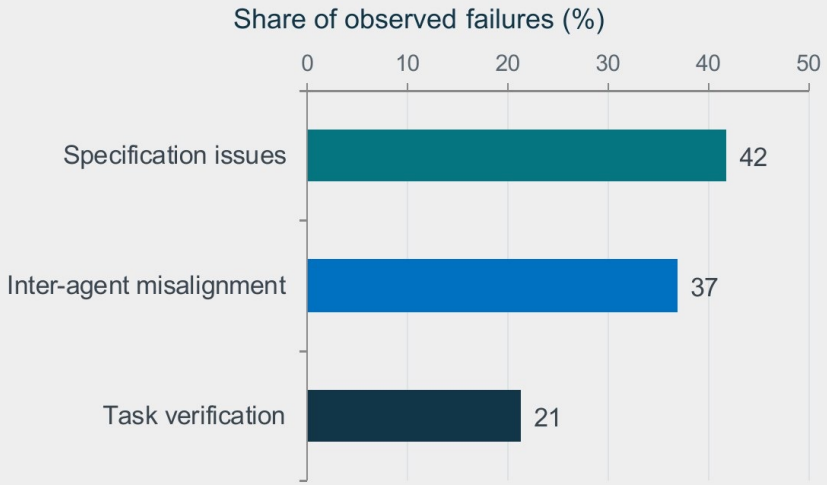
Key findings

- Model capability is the main driver of task success.
- A high coordination score does not guarantee a high task score — measure both.
- In the research scenario, the graph (peer-to-peer) topology performed best and the tree worst.
- Self-evolving planning improved milestone achievement by about 3%.

Source: Zhu et al. (2025), "MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents", ACL 2025.

↩ Link back: Agent Interactions lecture — centralized vs. distributed coordination

Why LLM Multi-Agent Systems Fail: MAST



Study: 200+ conversation traces from 7 open-source multi-agent frameworks, annotated by 6 experts → 14 failure modes.
Automatic labelling: an LLM-as-a-judge pipeline agreed with experts at Cohen's κ = 0.77.

Specification issues

Step repetition (17.1%) · Disobey task specification (11.0%) · Unaware of stopping conditions (9.8%)

Inter-agent misalignment

Reasoning–action mismatch (14.0%) · Fail to ask for clarification (11.7%) · Task derailment (7.2%)

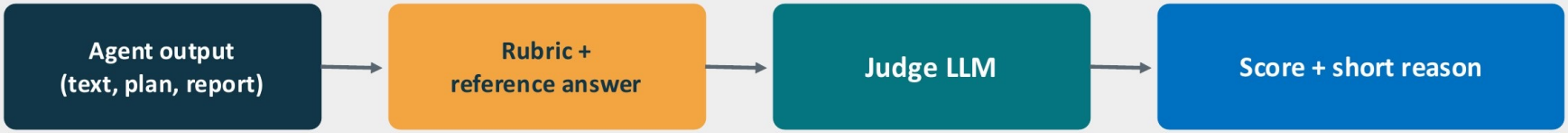
Task verification

Premature termination (7.8%) · No or incomplete verification (6.8%) · Incorrect verification (6.7%)

Source: Cemri, Pan, Yang et al. (2025), "Why Do Multi-Agent LLM Systems Fail?", UC Berkeley, arXiv:2503.13657.

↩ Link back: Agent Communication & Interactions — coordination failures

LLM-as-a-Judge: Scoring Open-Ended Outputs



Evidence it works

- Strong judges agreed with human preferences over 80% of the time — about the same as humans agree with each other.
- Scales to thousands of outputs at low cost.

Known biases — control them

- Position bias: prefers the first answer → swap order.
- Verbosity bias: prefers longer answers → rubric on content.
- Self-enhancement: prefers its own model → use a different judge.
- Always validate on a human-labelled sample.

Source: Zheng et al. (2023), "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena", NeurIPS 2023 Datasets & Benchmarks.

OpenTelemetry Tracing for AI Agents

One user request → one trace (span tree)

```
invoke_agent  Coordinator
  chat  (plan the task)
  invoke_agent  Researcher
    execute_tool  web_search
    chat  (summarize sources)
  invoke_agent  Reviewer
    chat  (check against rubric)
```

Standard attributes on each span

Attribute	What it gives you
`gen_ai.operation.name`	invoke_agent, execute_tool, chat …
`gen_ai.agent.name`	which agent did the work
`gen_ai.tool.name`	which tool was called
`gen_ai.usage.input_tokens` `gen_ai.usage.output_tokens`	cost of each step
span start / end time	latency of each step

Status: the GenAI conventions are still in development, but are already supported by major agent frameworks and tracing tools.

Source: OpenTelemetry (2025–2026), Semantic Conventions for Generative AI — agent and framework spans, github.com/open-telemetry/semantic-conventions-genai.

↩ Link back: Part 3 — traces & spans; JADE Sniffer, now for LLM agents

How Do We Evaluate LLM Agents?

Test success, consistency, cost, teamwork, and failure

1 Use realistic benchmarks
AgentBench · GAIA · SWE-bench · τ-bench · MultiAgentBench

2 Repeat the trials
pass@k: at least one success
pass^k: all k trials succeed

3 Report accuracy and cost
Compare systems on a Pareto frontier

4 Evaluate the team
Task score · Milestone KPI · Coordination score

5 Diagnose failures (MAST)
Specification 42% · Inter-agent misalignment 37% · Task verification 21%

6 Judge and trace outputs
Use code checks, human review, or an LLM judge; validate judges on human-labelled samples. Trace agent steps, tool calls, latency, and token use.

A Modern Evaluation Pipeline in Practice

Dataset	Traced runs	Evaluators	Experiments	Monitoring
20–100 test tasks with expected outcomes	every run recorded as a trace	code checks · LLM judge · human review	compare versions side by side	online evaluation in production

Example tool: LangSmith (used in the CAI 461 evaluation lab)

- Tracing of LLM, tool and agent steps with latency and token cost.
- Datasets and experiments with code-based, LLM-as-a-judge and pairwise evaluators.
- The same pipeline works with other OpenTelemetry-compatible tools — the method matters more than the tool.

Source: LangChain (2026), LangSmith documentation — Evaluation concepts, docs.smith.langchain.com.

↩ Link back: Topic 8 practical — instrumenting, tracing and evaluating a MAS

From Runs to Reliable Evaluation

A practical pipeline for multi-agent systems

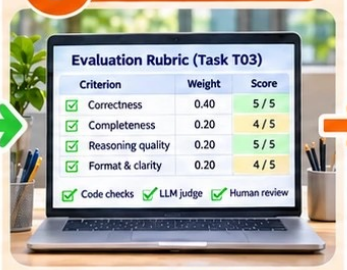
1 Dataset

20–100 tasks with expected outcomes

2 Traced runs

Record every agent and tool step

3 Evaluators



Code checks · LLM judge · Human review

4 Experiments

Compare versions side by side

5 Monitoring

Evaluate the system in production

CAI 461 Evaluation Lab

SET UP

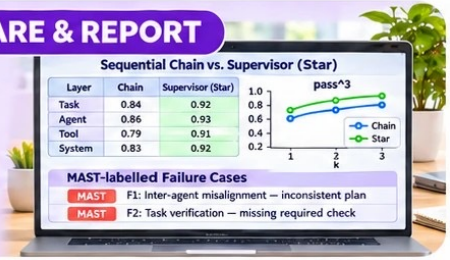
Instrument a coordinator + 2 workers. Prepare **20 test tasks.**

RUN

Use exact-match, rubric, latency and token-cost evaluators. Run ***k* = 3** trials; report success rate and **pass^3.**

COMPARE & REPORT

Sequential chain vs. supervisor (star). Present a four-layer table, one chart, and **two MAST-labelled** failure cases.



Princess Nourah Bint
Abdulrahman University

Part 6

Bringing It Together

A four-layer evaluation framework and a case activity

Four-Layer Evaluation of Agent-Based Systems
1. Individual Agent
Did each agent achieve its task?
Measures:
Performance measure · Expected utility · Task success
2. Interaction
Do agents work well together?
Measures:
Social welfare · Pareto efficiency · Stability · Price of anarchy
Multi-Agent System
Campus Operations Hub
3. System
Does the whole system perform well at scale?
Measures:
Latency · Throughput · Scalability · Robustness
4. Trust & Deployment
Can people rely on it in real use?
Measures:
Reliability (pass^k) · Cost · Safety and policy violations · Failure modes
Trustworthy AI Operations
Reliability (pass^k)
Cost
Safety and policy violations
Failure modes

How to Design a Sound Evaluation Experiment

Plan, run, and report a fair multi-agent comparison

PLAN

Hypothesis Make a specific, testable prediction.

Variables **Independent:** protocol and number of agents (*n*). **Dependent:** latency and success rate. **Controlled:** tasks and hardware.

Baseline Compare with random assignment, a greedy method, or a single agent.

RUN

Repeat Run each setting 30 times with different random seeds.

Keep conditions consistent Use the same tasks and hardware across comparisons.

REPORT

Statistics Report mean ± standard deviation or a 95% confidence interval; test for differences.

Reproducibility Publish code, parameters, seeds, and model versions.

EXAMPLE HYPOTHESIS | **Contract Net completes tasks faster than random assignment for $n \geq 20$.**

Case Activity: Design an Evaluation Plan

Case A — PNU Smart Shuttle

8 shuttle agents bid for ride requests across campus using Contract Net.

Case B — Campus Evacuation Model

A NetLogo model of 3,000 students leaving buildings during a fire drill.

Case C — Course-Advising Crew

Three LLM agents (planner, retriever, checker) answer students' registration questions.

Each group delivers — one slide or one chat message

1. One metric from each of the four layers
2. One baseline to compare against
3. One experiment: what you vary, how many runs
4. What you will trace or log
5. One likely failure mode and how you would detect it

8 minutes in breakout rooms · 4 minutes for two groups to share

Lab Preview: Instrument, Trace, Evaluate

1 Instrument

Enable tracing on a small multi-agent system (coordinator + 2 workers).

2 Build a dataset

Write 20 tasks with expected outcomes.

3 Define evaluators

Exact-match check, LLM-judge rubric, latency and token cost.

4 Run k = 3 trials

Compute success rate and pass^3 for each task.

5 Compare designs

Sequential chain vs. supervisor (star) topology.

6 Report

Four-layer table, one chart, two failure cases labelled with MAST.

↩ Link back: Course specification, Topic 8 practical (2 hours)

Key Takeaways

1. Decide what “good” means first: measure the environment, not the agent’s behaviour.
2. Expected utility and success probability are the foundation — modern benchmarks still estimate them.
3. For societies of agents, report welfare and stability together; the price of anarchy shows the cost of selfishness.
4. Measure curves, not points: scalability and robustness need repeated, controlled experiments.
5. For models, verify, validate and replicate — and report with ODD.
6. For LLM agents, add reliability (pass^k), cost, failure modes and validated judges — all built on tracing.

References — Course Textbooks

1. Wooldridge, M. (2009). An Introduction to MultiAgent Systems (2nd ed.). John Wiley & Sons. — Ch. 2, 8, 11; Part III.
2. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. — Ch. 2, Ch. 18.
3. Shoham, Y., & Leyton-Brown, K. (2008). Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge University Press. — Ch. 3, §6.4.
4. Wilensky, U., & Rand, W. (2015). An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo. MIT Press. — Ch. 6, Ch. 7.
5. Bellifemine, F. L., Caire, G., & Greenwood, D. (2007). Developing Multi-Agent Systems with JADE. John Wiley & Sons. — JADE tools.
6. Smith, R. G. (1980). The Contract Net Protocol: High-level communication and control in a distributed problem solver. IEEE Transactions on Computers, C-29(12), 1104–1113.
7. Koutsoupias, E., & Papadimitriou, C. (1999). Worst-case equilibria. Proc. STACS 1999, LNCS 1563, 404–413.
8. Grimm, V., et al. (2006). A standard protocol for describing individual-based and agent-based models. Ecological Modelling, 198(1–2), 115–126.
9. Grimm, V., et al. (2020). The ODD protocol for describing agent-based and other simulation models: A second update. JASSS, 23(2), 7.
10. FIPA (2002). FIPA ACL Message Structure Specification, SC00061G.

References — Recent Research and Standards

11. Liu, X., et al. (2024). AgentBench: Evaluating LLMs as Agents. ICLR 2024.
12. Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2024). GAIA: A Benchmark for General AI Assistants. ICLR 2024.
13. Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.
14. Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2025). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. ICLR 2025.
15. Zhu, K., et al. (2025). MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. ACL 2025.
16. Cemri, M., Pan, M. Z., Yang, S., et al. (2025). Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657.
17. Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2025). AI Agents That Matter. Transactions on Machine Learning Research.
18. Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks.
19. OpenTelemetry. Semantic Conventions for Generative AI (agent and framework spans). github.com/open-telemetry/semantic-conventions-genai.
20. LangChain. LangSmith Documentation — Evaluation. docs.smith.langchain.com.

Princess Nourah Bint
Abdulrahman University

THANK YOU!

Questions and Discussion

Prof. Dr. Mohd Shahizan Othman · Universiti Teknologi Malaysia
Dr. Eman Aljabarti · Princess Nourah bint Abdulrahman University